



Orion Technical Assessment Documentation

TABLE OF CONTENTS

1. [Dataset Overview\(...click here\)](#)
2. [Data Quality Issues\(...click here\)](#)
3. [Data Transformations\(...click here\)](#)
4. [Explaining ETL logic & Workflows\(...click here\)](#)
5. [Data Modeling & Architecture\(...click here\)](#)

Dataset Overview (Sales Data)

Total Rows	Total Columns
298,246	18

Data Quality Issues

1. Product Name Structure Issue

- The **Product Name** column is not a simple product name. It is a structured field containing:
Brand + Product Description + Product Code + Color
- This makes it unsuitable for direct analysis and requires parsing into separate features.

2. Column Naming Inconsistency

- Some column names were inconsistent and not suitable for:
 - Python processing
 - SQL queries
 - Dashboard visualization
- Standardization Applied

Before	After
Product Name	ProductName
Customer Code	CustomerCode
Net Price	Net Price

3. Incorrect Color Values

- The **Color** column initially contained incorrect values, showing brand names instead.
- This issue was fixed by extracting the correct color directly from the Product Name field.

4. Customer Code Formatting Issue

- Some values in **Customer Code** were missing the required prefix (**CS**).
- This was corrected by standardizing all values to ensure consistent formatting.

5. Customer Name Formatting Issue

- The original format of **names** was inconsistent for dashboard usage.
- Adjusted name formatting for dashboard compatibility:
Transformed the original **Last Name + First Name** structure into a standard **First Name + Last Name**

6. OrderDate Data Type Issue

- The **OrderDate** column was stored as **object** type.
- It was converted into **datetime** format to enable:
 1. Time-based analysis
 2. Feature engineering (Year, Month, Quarter, etc.)

7. Columns with Missing Data:

Column Name	Number of Missing Data
Name	29,797
Education	29,797
Occupation	29,797

- Large number of missing values compared to total rows (~90% missing)
- This is a significant data quality issue and requires careful handling (imputation or exclusion depending on analysis goals)

8. Duplicate Records

- Number of duplicate rows is: **218008 rows**
- After dropping the missing data, no duplicate rows were found therefore, no further duplicate removal was required

9. If the data will be used for Machine Learning, you also need to check for outliers and handle imbalanced data

Data Transformations

1. Drop duplicate rows

2. Drop missing data

- First, I need to investigate whether the customer data is missing from the source itself or due to an incorrect transformation. However, in this case, I do not have access to the data source to determine the root cause, so I decided to drop the data since 90% of it is missing

3. Rename columns to make names consistent and easier to use

4. Convert **OrderDate** to **datetime**

5. Create Dynamic Date Table Generation

- In multi-source data platforms, analyzing transactional facts (**Fact_Sales** and **Fact_Forecast**) across disparate chronological granularities requires a centralized, continuous time dimension table (**Dim_Date**).
- The date table is dynamically bounded: it begins on the first day of the first month of the earliest year in the dataset and concludes on the last day of the last month of the latest year.

6. Extract time features [**Year** , **Month** , **Day** , **Quarter**]

7. Create SalesAmount feature

- **SalesAmount** = **Quantity** * **NetPrice**

8. Extract structured parts from **Product Name**

- We attempted to extract structured features from the 'Product Name' column by splitting it into:

- ProductName (Product Description)
- ProductCode
- Color

- **Challenges Faced:**

1) Inconsistent Product Code Formats

- Product codes were not uniform. This caused regex patterns like `[A-Z]\d+` to fail.

E175
M310
X100
XD233

2) Inconsistent Product Name Structure

- Not all product names followed the same format:
 - Some included dates (2009)
 - Some included punctuation (PC2.33)
 - Some had extra descriptive text
 - Some lacked color values

3) Color Was Not Always Reliable

- Color sometimes existed (Black, Silver, Brown)
- Sometimes missing entirely
- Sometimes confused with other words in the description

I successfully resolved these data quality issues and extracted clean, structured data by applying customized, flexible Regex patterns to each column individually

9. Dropping new nulls because some products lacked **color** info within the Product Name during the extraction process

10. Standardize Customer Code to always start with 'CS.'

11. Adjusted name formatting for dashboard compatibility:

Transformed the original **Last Name + First Name** structure into a standard **First Name + Last Name**

Explaining ETL logic & Workflows

- Each step of the ETL process is encapsulated within a dedicated Python function to ensure maintainability, code reusability, and clean error handling.
- The data modeling phase is executed directly within the building pipeline, relational **Fact and Dimension tables** and mapping their constraints using **SQLAlchemy**.
- To guarantee pipeline resilience, every functional block is wrapped in robust **try-except** structures, ensuring comprehensive error logging and preventing silent pipeline failures.

- **Modular ETL Architecture: Function-Level Breakdown(**Sales Pipeline**):**

Function Name	Responsibilities
<code>extract_data()</code>	Responsible for the Extraction phase . Reads the raw data from the source file (JSON) and loads it into a Pandas DataFrame to prepare it for the transformation step.
<code>transform_data()</code>	Responsible for the Transformation phase . Cleans the data by removing duplicates and missing values, standardizes column names, converts data types, creates new features like SalesAmount, extracts ProductCode and Color from ProductName, formats customer data, and Create Dynamic Date Table Generation.
<code>data_validation()</code>	Responsible for Data Quality Checks . Validates the transformed data by checking missing values, duplicate records, and data types to ensure the data is ready before loading into the database.
<code>reset_tables()</code>	Responsible for Database Reset . Clears existing data from tables before reloading new data to avoid duplicate records and maintain a clean database state while respecting table dependencies. Production Consideration: For production workloads, it is better to use incremental loading instead of resetting tables. New and updated records should be handled using Upsert/Merge logic to maintain data consistency and improve pipeline performance.
<code>create_dims()</code>	Responsible for creating the Dimension Tables in the database. Creates tables like <code>dim_product</code> , <code>dim_customer</code> , <code>dim_geography</code> , and <code>dim_date</code> with the required structure and relationships.
<code>create_fact()</code>	Responsible for creating the Fact Table . Creates the <code>fact_sales</code> table and defines the relationships with dimension tables using foreign keys to build the star schema model .
<code>load_data()</code>	Responsible for the Load phase . Loads transformed data into staging tables, select dimension column from cleaned data, inserts data

	into dimension tables, and populates fact tables by joining staging data with dimensions. Also saves output datasets as CSV files.
run_pipeline()	Responsible for Pipeline Orchestration . Controls the full ETL workflow by executing extraction, transformation, validation, database setup, and loading steps in the correct order, while handling pipeline errors.

- **Modular ETL Architecture: Function-Level Breakdown(**Forecast Pipeline**):**

-

Although the current raw source data for the forecast track is clean and structured, the transformation pipeline is explicitly architected with defensive programming principles. We designed dedicated transformation functions not just for the present state, but to enforce strict schema governance, data type validation, and constraint mapping for any future data streams entering the pipeline. This ensures that upstream changes or automated future uploads will scale seamlessly without breaking the data model or downstream dashboards.

Function Name	Responsibilities
extract_data()	Responsible for the Extraction phase . Reads the raw data from the source file (JSON) and loads it into a Pandas DataFrame to prepare it for the transformation step.
transform_data()	Responsible for the Transformation phase . Cleans the data by removing duplicates and missing values, converts data types.
data_validation()	Responsible for Data Quality Checks . Validates the transformed data by checking missing values, duplicate records, and data types to ensure the data is ready before loading into the database.
reset_tables()	Responsible for Database Reset . Clears existing data from tables before reloading new data to avoid duplicate records and maintain a clean database state while respecting table dependencies. Production Consideration: For production workloads, it is better to use incremental loading instead of

	resetting tables. New and updated records should be handled using Upsert/Merge logic to maintain data consistency and improve pipeline performance.
<code>create_dims()</code>	Responsible for creating the Dimension Tables in the database. Creates tables like <code>dim_brand</code> with the required structure and relationships.
<code>create_fact_forecast()</code>	Responsible for creating the Fact Table . Creates the <code>fact_forecast</code> table and defines the relationships with dimension tables using foreign keys to build the star schema model .
<code>load_forecast_data()</code>	Responsible for the Load phase . Loads transformed data into staging tables, select dimension column from cleaned data, inserts data into dimension tables , and populates fact tables by joining staging data with dimensions . Also saves output datasets as CSV files.
<code>run_pipeline()</code>	Responsible for Pipeline Orchestration . Controls the full ETL workflow by executing extraction, transformation, validation, database setup, and loading steps in the correct order, while handling pipeline errors.

Data Modeling & Architecture

- **The Star Schema Model:**
 - The data warehouse architecture is built upon a **Star Schema** design pattern.
- **Granularity Analysis & Mismatch Handling :**

One of the core architectural challenges solved in this project was managing **Granularity Mismatch** between disparate business tracks

- **Fact_Sales(High/Atomic Granularity):** Captures data at the lowest operational level **one row per sales transaction line**. This enables deep-dive diagnostics and atomic-level slicing.

- **Fact_Forecast(Low/Macro Granularity):** Captures strategic data at a highly aggregated level **one row per Country / Brand / Year** (specifically constrained to 2009 targets).

The problems:

Directly joining these two tables or linking them through mismatched attributes would introduce severe logical flaws, such as the **Fan-out effect (Data Duplication)** and massive **Double-Counting** of forecast numbers in BI dashboards.

The Solution:

Conformed Star Schema Design To bridge the gap between the mismatched grains without introducing **fact-to-fact relationships**, a Conformed Star Schema architecture was implemented using **shared** and **bridging** dimensions.

1- Bridging Mismatched Products via Dim_Brand

- **Fact_Forecast** does not contain individual product data; it targets the overall **Brand** level.
- A dedicated **Dim_Brand** dimension was created. **Fact_Forecast** connects directly to **Dim_Brand** via **BrandKey**. **Dim_Product** references **Dim_Brand** as a lookup.
- This establishes a clean **Bridge**, allowing both actual sales and forecast metrics to be sliced and compared consistently at the Brand level.

2- Resolving the Geography Fan-out (Dim_Geography vs. Country Grain)

- **Dim_Geography** contains micro-level data (multiple States and Cities per Country). Joining **Fact_Forecast** (Country grain) directly to this table on a loose country name string would duplicate the forecast rows for every matching city/state combo.
- **Resolution:** To enforce dimensional integrity and eliminate the fan-out threat, a dedicated **Dim_Country** table was designed as a Conformed Dimension.
- **The DAX** extracts the unique **CountryRegion** values from **both Fact_Sales and Fact_Forecast**, combines them using a **UNION** operation to eliminate duplicates, and generates a unified master list of countries with a surrogate **CountryKey**.
- **Relationship** Both **Fact_Sales** and **Fact_Forecast** are then structurally connected to this shared **Dim_Country** table. This completely neutralizes the data duplication risk while ensuring seamless cross-filtering.

- Schema Structural Breakdown

Table Name	Table Type	Key Columns	Primary Key (PK)	Foreign Key (FK)
fact_sales	Fact Table	Saleskey , productkey, customerkey, geographykey, quantity, Netprice, salesamount	Saleskey	productkey, customerkey, geographykey
fact_forecast	Fact Table	forecastkey , yeardatekey , year , brandkey, countryregion, forecast	forecastkey	yeardatekey , brandkey, countryregion
dim_product	Dimension	productkey , productname , productcode , brand , color , category , subcategory	productkey	brandKey
dim_customer	Dimension	customerkey , customercode , name , education , occupation	customerkey	
dim_geography	Dimension	geographykey , continent , countryregion , state , city	geographykey	countryregion
dim_date	Dimension	datekey , date , day , month , year , quarter	datekey	
dim_brand	Bridge Dimension	brandKey , brand	brandKey	
dim_country	Bridge Dimension	countryKey , countryRegion	countryKey	

- Relationships

From	To
Dim_Date[DateKey]	Fact_Sales[DateKey]
Dim_Date[DateKey]	Fact_Forecast[YearDateKey]
Dim_Product[ProductKey]	Fact_Sales[ProductKey]
Dim_Customer[CustomerKey]	Fact_Sales[CustomerKey]
Dim_Geography[GeographyKey]	Fact_Sales[GeographyKey]
Dim_Country[countryregion]	Dim_Geography[countryregion]
Dim_Country[countryregion]	Fact_Forecast[countryregion]
Dim_Brand[BrandKey]	Fact_Forecast[BrandKey]
Dim_Brand[brand]	Dim_Product[brand]

- Data Modeling Diagram

